

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

UC MATH 600 — Mathematics for Computer Science

Proof is a tool for being sure, and this is the course that makes the rest of the program readable

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Prerequisite · take before Semester 1

Prerequisites: Secondary school algebra. Nothing else is assumed, and nothing in this course requires calculus

Effort: 8–10 hours per week · 13 modules + 2 project weeks

Deliverable: A worked proof portfolio — thirty proofs you wrote yourself, including at least five you got wrong first and the record of what was wrong with them

Course description

This is one of the two mathematics prerequisites, and skipping it is the single most common reason Semester 1 fails. CSCE 629 is unreadable without proof technique, induction, and asymptotic reasoning — not difficult, *unreadable*, because its arguments are written in a language this course teaches.

The first third is proof itself. Module 01 argues that a proof is a tool rather than a ritual, and Modules 02 and 03 give you the three techniques that cover almost everything — direct, contrapositive, contradiction, and then induction, which is the one computer science uses most and the one most often done badly.

The second third is the objects. Sets, functions, relations, modular arithmetic, counting, and graphs (Modules 04 through 08) — which are the vocabulary every later course assumes you already have, and which it will not stop to define.

The third is the quantitative part. Asymptotics and recurrences (Modules 09 and 10) are what CSCE 629 is actually about; discrete probability and concentration (Modules 11 and 12) are what CSCE 658 and every machine learning course rest on.

And the closing module is about honest claims, which is the rule this whole program is built on. A proof with a gap is not a short proof — it is a different thing — and learning to see your own gaps is most of what this course is for.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain what a proof establishes and what it does not.
- 2 Write direct, contrapositive, and contradiction proofs.
- 3 Write induction proofs, including strong and structural.
- 4 Work fluently with sets, functions, and relations.
- 5 Use modular arithmetic and the division algorithm.
- 6 Count correctly, including with overcounting corrections.
- 7 Model problems as graphs and reason about them.
- 8 Prove properties of trees and connectivity.
- 9 Use asymptotic notation precisely.
- 10 Solve recurrences, including by the master theorem.
- 11 Compute discrete probabilities without the standard errors.
- 12 Use expectation, linearity, and concentration bounds.
- 13 State what you proved, and find your own gaps.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What a Proof Is For	Why write it out when you already believe it?
2	Logic and Proof Technique	You are stuck on a proof. What do you try first?
3	Induction	Why is it enough to prove one case and one step?
4	Sets, Functions, Relations	What does it mean for two infinite sets to be the same size?
5	Number Theory and Modular Arithmetic	Why does the clock arithmetic you learned as a child underlie public-key cryptography?
6	Counting	How many ways, and how do you know you counted each once?
7	Graphs	What problems become easy once you draw them as a graph?
8	Trees and Connectivity	Why does a tree on n vertices always have $n - 1$ edges?
9	Asymptotics	What does $O(n \log n)$ mean, precisely?

Wk	Module	Core question it answers
10	Recurrences	$T(n) = 2T(n/2) + n$. Now what?
11	Discrete Probability	The test is 99% accurate and you tested positive. What is the probability you have it?
12	Expectation and Concentration	How far from its average can a random variable get?
13	Claiming You Proved Something	Is that a proof, or is it a sketch you believe?

Projects

Project 1 — Fifteen proofs, and five of them wrong (after Module 6)

Build the first half of the portfolio, and keep the failures.

Requirements

- **Fifteen proofs written out in full**, from Modules 02 through 06 — **at least three by each of direct, contrapositive, contradiction, and induction**.
- **Five of them attempted wrongly first**, with **the wrong version kept and the error named** — which is the part that teaches.
- **One conjecture tested computationally before proving**, with the test code included.
- **And one conjecture disproved by counterexample**, found rather than looked up.

Done means

- **The wrong versions kept and the errors named** — **which is the actual deliverable**; a portfolio of fifteen correct proofs with no history shows less.
- **Each proof stating what it assumes** at the top, which is the habit Module 13 asks for.
- **The induction proofs having an explicit base case and an explicit statement of the inductive hypothesis** — omitting either is the characteristic error (Module 03 §3).
- **And the counterexample genuinely found**, with the search described.

Project 2 — Fifteen more, and the asymptotic ones (after Module 12)

Finish the portfolio with the material CSCE 629 will assume.

Requirements

- **Fifteen further proofs** from Modules 07 through 12, **including at least three asymptotic bounds and three probability arguments**.
- **Three recurrences solved three ways** — by expansion, by the master theorem, and by substitution-and-induction — **with the three answers agreeing**.
- **One counting problem solved two ways**, with the two answers shown to be equal algebraically.
- **One graph result proved** and then verified computationally on a hundred random graphs.
- **And a self-audit: go back to Project 1's fifteen proofs and find at least two gaps you did not see at the time**.

Done means

- **The self-audit finding real gaps** — **which it will**, and **a self-audit reporting none is the one outcome that fails this project** (Module 13 §2).
- **The three recurrence methods agreeing**, with any disagreement tracked down rather than averaged.

- **The double-counting identity proved both ways**, which is the clearest demonstration that counting is reasoning rather than arithmetic.
- **And the asymptotic proofs using the definition — constants exhibited, threshold stated** — rather than appealing to intuition about growth (Module 09 §2).

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Lehman, Leighton & Meyer — Mathematics for Computer Science (free book)	The primary text, free in full. This course follows its development closely, and its exercises are the natural problem set. It is written for exactly this audience.
MIT 6.042J / 6.1200J Mathematics for Computer Science (OCW, full video)	The lectures, free. Watch them alongside the modules — the recitation videos are where the proof technique is actually demonstrated.
Velleman — How to Prove It	Modules 01 through 03. The best book on proof technique as a skill rather than a subject, and it is unusually patient. Library copy.
Rosen — Discrete Mathematics and Its Applications	Modules 04 through 08 as a reference. Comprehensive and dry; good for looking something up, poor for reading straight through. Library copy.
Mitzenmacher & Upfal — Probability and Computing	Modules 11 and 12. Where discrete probability is developed for computer scientists rather than for statisticians, and the direct prerequisite for CSCE 658. Library copy.
Cormen, Leiserson, Rivest & Stein, chapters 3 and 4	Modules 09 and 10. Asymptotics and recurrences in the form CSCE 629 will use them. Library copy.

Tooling

- **Paper, and a great deal of it. This is the one course in the program with no code requirement**, and writing proofs by hand is not nostalgia — it is slower, which is the point.
- **A proof portfolio file**, kept from week one — **which is the deliverable**, and which is far easier to assemble continuously than retrospectively.
- **Python, for checking — a conjecture you can test on ten thousand cases before trying to prove it is a conjecture you waste much less time on**, and finding a counterexample in four lines is a legitimate result (Module 01 §3).
- **And somebody to read your proofs**, ideally — **because the gap you cannot see is the whole difficulty** (Module 13 §2). Failing that, read them aloud a day later, which catches a surprising amount.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.